SECOND REVIEW · ACADEMIC PROTOTYPE · NOT AN OPERATIONAL SYSTEM

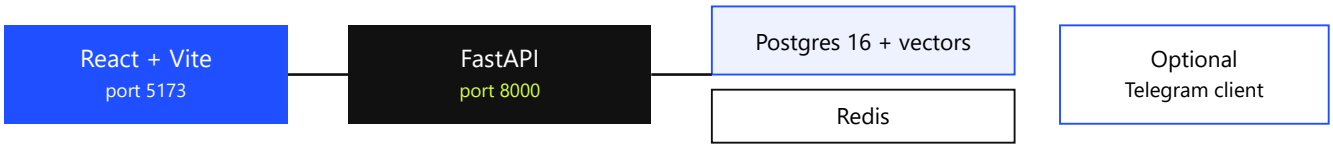
Technology stack, data, and build notes

What the prototype is built from, where the posts come from, and what broke along the way.

This brief is shared. It sits beside the five agent briefs. It describes an undergraduate prototype. It is not an operational intelligence system, and it is not deployed by any agency. The public problem statement is the only link to that origin.

1. How the running system is split

The laptop runs the website and the API. Docker runs only the two data services. Models are loaded inside the API process, one at a time. A live Telegram client is optional and is not required for the review.



The browser talks only to the API. The API talks to Postgres. Redis is there for background jobs.

Figure 1. The website never opens the database itself. Collection, analysis, and Ask all go through the API.

2. The stack, and the job each part does

Layer	What we use	What it does in this project
Language	Python 3.11	API, collection, models, and the offline seed.
API	FastAPI, Pydantic v2, Uvicorn	Seven data routes plus health. Request bodies are checked before they touch the database.
Auth	HTTP Basic	The dashboard sends a lab username and password. The health check stays open so we can see whether the process is up.
Database	PostgreSQL 16, SQLAlchemy 2.0, Alembic	Channels, posts, analysis rows, and Ask logs. Migrations create the tables. The first migration also enables the vector extension.
Search	pgvector	Stores 384-number embeddings. Ask rejects a question when the nearest summary is farther than cosine distance 0.65.
Jobs	Celery, Redis 7	Designed for a slow background cycle: one channel at a time, one ingestion worker. The review button can also run the same analysis immediately.
Containers	Docker Compose	Postgres uses the `pgvector/pgvector:pg16` image. Redis uses `redis:7-alpine`. The API and the website run on the host, not inside those containers.

Collection	Pyrofork	Optional live reader for public Telegram channels and groups. A saved session file is a single SQLite file, so only one client may use it.
Reading posts	PyTorch, Hugging Face Transformers	Three published models, loaded one after another: sentiment, emotion, then stance.
Themes and Ask search	sentence-transformers, BERTopic, HDBSCAN, UMAP	The same MiniLM model embeds posts for themes and embeds the short summaries Ask searches.
Graph	NetworkX, python-louvain	Directed interactions, then an undirected copy for communities. The score is an equal average of three signals.
Written answers	Local Ollama, or an online model when a key is set	Search always happens in Postgres first. The language model only writes from the passages that passed the cutoff. Every question and the passages used are logged.
Website	React 18, Vite 6	Seven views. Styling is our own CSS. The dev server proxies `/api` to port 8000.

> The early design notes mention Tailwind, Recharts, and a force-graph library. Those were not added. The pages draw their own bars and the network canvas. Say that if a reviewer has the older note.

3. The models, and why they are not all loaded together

Job	Checkpoint	Rough weight
Sentiment	`cardiffnlp/twitter-xlm-roberta-base-sentiment`	about 1.1 GB
Emotion	`SamLowe/roberta-base-go_emotions`	about 500 MB
Stance	`valhalla/distilbart-mnli-12-3`	about 260 MB
Embeddings	`paraphrase-multilingual-MiniLM-L12-v2`	about 470 MB
Offline answer model	Ollama `llama3.2:3b`, 4-bit	about 2 GB

The reference card is a GTX 1650 Ti with 4 GB. Those weights do not fit together. The pipeline loads one model, finishes the batch, frees the card, then loads the next. We did not train any of them. The large BART stance model, about 1.6 GB, is a report comparison only and is not loaded in the demo. If the card is unavailable, the same code runs on the CPU. The labels do not change. The run takes longer, so we do not promise a 15–25 second demo on CPU or on thousands of posts. That timing is only for a batch of at most 50 posts on the reference GPU.

4. Where the posts come from

There are three sources. They all land in the same tables. A repeated message is recognised by its channel and message id, so a second import does not create a second row or wipe an analysis that already ran.

Source	Command	When we use it

Offline seed	`python -m ingestion_a.backfill --offline`	The review and any room without a stable network. Prepared posts are given recent times so the 48-hour pages are not empty. Extra posts can be generated from written templates. No Telegram call is made.
Desktop export	`python -m ingestion_a.import_exported`	Chats exported from Telegram Desktop as HTML (`messages.html`) or JSON (`result.json`). The importer reads the channel name, message id, text, time, and reaction counts.
Live reader	`python -m ingestion_a.download_latest`	Optional. Public channels and groups whose titles match the project subjects: geopolitics, strategy, conflict, defence, OSINT, space, cybersecurity, politics, governance, and development. Latest 80 messages. Photos only, saved under `Telegram Desktop`.

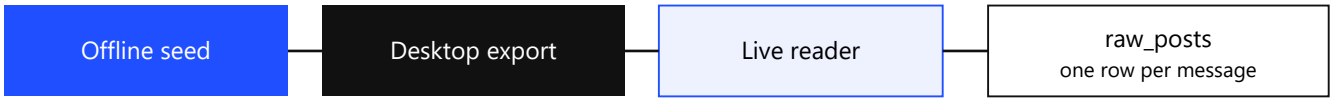


Figure 2. Three doors, one table. The time used later is the time on the message, not the time we stored the row.

The live reader keeps a title only when it matches the subject list. Unrelated exports — books, films, job posts, cracked software — are left out. A title written in decorative Unicode is folded to plain letters before that check, otherwise a real channel can look like a random string. The reader sleeps when Telegram asks it to wait, then continues the same chat. The live pull was paused before every matching chat had been saved. The review does not depend on finishing it.

What a stored post holds: text, the message time, view count, reaction counts, a photo path when a picture was saved, and a status of pending or processed. Display names and bios are stored only when a person actually spoke in a public discussion. Subscriber lists are not available, and private one-to-one chats are not the collection design. After the exam, `scripts/post_viva_scrub.sql` clears message text, display names, and bios.

5. Challenges, and what we changed

What happened	Why it mattered	What we did
All models together exceed 4 GB	The card runs out of memory and the demo dies.	Load one model, free it, then load the next. Stance stays on the small DistilBART checkpoint.
The GPU was not always the device in use	A CPU run of hundreds of posts is far slower than the 15–25 second GPU budget.	The review button still caps a live trigger at 50 posts. We do not quote that timing for a full archive or for CPU.
Telegram FloodWait	A fast loop makes Telegram freeze the account, sometimes for most of an hour.	One client, a pause between chats, and a sleep for the exact wait Telegram returns. The pull can be stopped and resumed later.
The native Telegram cipher would not build	`tgcrypto` needs a C compiler this machine did not have.	Pyrofork fell back to its pure-Python cipher. Collection is slower. It still saves messages.

Two clients, one session file	The session is SQLite. A second process locks or corrupts it.	Ingestion is limited to one worker. We do not start a second download while one is open.
Campus Wi-Fi and missing keys	A review that needs Telegram or a cloud model fails in the room.	The offline seed is the product. Live Telegram and an online answer model are optional.
Decorative channel titles	A keyword test on the raw title skipped channels that were in scope.	Titles are folded to plain letters before the subject check.
A short word matched the wrong title	A fragment such as “war” inside an unrelated name kept a chat we did not want.	Blocked words are matched as whole words, and a block list wins unless a title is an explicit keep.
The same audience card appeared three times	Each analysis run inserted a new snapshot of the same group.	The page keeps the newest snapshot of each group label.
New downloads did not appear on Sentiment	That page only shows posts that already have a result, inside 48 hours.	New rows stay pending until Component B runs. The next run reads the newest message times first. A photo with no words is skipped, because there is no text to label.
Real groups were smaller than 50 accounts	The privacy rule hides a group under 50. A strict page would look empty.	The research line stays 50. A demo run may show a smaller card, and the speaker has to say it is below the line.
Ask can sound confident with no evidence	A language model will invent a theme if it is asked to answer from nothing.	Search first. If the nearest summary is beyond distance 0.65, Ask stops. The model never sees the raw archive, only the short summaries.

6. What we are not claiming

Locked choice	Why a reviewer should hear it
No accuracy percentage	The comparison benchmarks in Phase 5 have not been run.
No bot or campaign verdict	The network score is an equal average of three behaviour signals and a screen at 0.70.
No individual profile	Audience output is a group sketch from public text and posting hour.
No second product inside the dashboard	There is no PDF export page and no live socket. The page refreshes by ordinary HTTP.

> **One minute.** “Postgres holds the posts. Five agents read them: collection, then one reading per post, then audience, themes, and the graph. The website only calls the API. The exam path is a prepared seed, because Telegram rate limits and a 4 GB graphics card both failed us when we treated the live path as the product.”